

WhistleSense

Started 01/05/2026

Core idea

The product is a **smart pressure-cooker whistle counter** designed for Indian kitchens.

It is a compact, strap-mounted device that attaches securely to the pressure cooker handle. The device listens for pressure cooker whistles, senses cooker vibration, and monitors whether the cooker is actively hot. The user sets a target number of whistles, such as 2 or 3, and the device counts each confirmed whistle event.



Once the target whistle count is reached, the device alerts the user through an on-device buzzer, LED/display indication, and eventually a phone notification.

The core purpose is to help users stop manually tracking pressure cooker whistles, avoid overcooking food, and feel more confident stepping away from the kitchen while cooking.

This would be the first basic prototype later to automate the whole kitchen processes. Industrial automation in your kitchen is what we would be doing.

Customer Poll Questions

- “When was the last time you missed a whistle?”
- “What happened?”
- “How often does this happen?”
- “What do you do while the cooker is on?”
- “Would an alert-only version help?”
- “Would you pay ₹999, ₹1,499, or ₹2,499 for it?”
- “Would you trust it near your cooker?”
- “Would you attach it to the handle every time?”
- “Would your mother/mother-in-law use it?”
- “Would you still want it if it did not turn off the stove?”

The biggest question is : **Will people pay for alert-only, or do they only get excited when it can turn off the stove?**

WhistleSense V0 - Prototype

WhistleSense V0 is a temporary sensor band/clip attached to a pressure cooker handle that records real cooking sessions and helps us identify whistle patterns. It will not connect to the internet or send alerts to the user.

It should:

1. Sit on or around the cooker handle.
2. Measure sound, vibration, and temperature.
3. Let you manually mark each real whistle.
4. Save timestamped sensor data.
5. Show whether the device can detect whistles accurately.

Prototype success criteria

For the first prototype, success would be defined like this:

Good enough to continue

- detects 8 out of 10 real whistles
- false counts are understandable and fixable
- handle placement gives usable sensor data
- device survives one full cooking session safely

Strong signal

- detects 9 out of 10 real whistles
- false positives are rare
- works across at least 3 cooker types
- works with normal kitchen background noise

Not good enough

- vibration is random or weak across cookers
- microphone gets confused too often
- temperature is too slow to help
- users hate attaching it to the handle

Prototype requirements

1. Microcontroller

ESP32 dev board.

Pros:

- cheap
 - easy to program
 - supports Wi-Fi/Bluetooth later
 - enough for early signal processing
 - widely available in India
-

2. Microphone sensor

INMP441 I2S MEMS microphone.

Purpose:

- detect the pressure cooker whistle sound
 - measure audio intensity and frequency pattern
 - compare whistle vs mixer/TV/talking/noise
-

3. Vibration sensor

Use an **MPU6050 accelerometer/gyro** or similar.

Purpose:

- detect vibration bursts when the whistle happens
 - check whether vibration travels to the handle clearly
 - compare different cooker types
-

4. Temperature sensor

Use a **DS18B20, NTC thermistor**, or **TMP117**.

Purpose:

- detect that the cooker is actively heating
 - understand handle temperature profile
 - reduce false positives from random sounds
-

5. Manual whistle button

A simple push button that you press every time you hear an actual whistle.

Purpose:

- creates ground-truth labels
- helps compare real whistle timing vs sensor data
- lets you train the future algorithm

Without this, you will have sensor data but no clean way to know exactly what was a real whistle.

6. Buzzer and LED

Purpose:

- later live testing
 - debugging
 - simple alert when target count is reached
-

7. Data logging

This is the most important V0 feature.

Option A: Log to laptop over USB

Simpler.

The ESP32 sends data through serial to your laptop. Your laptop saves it as a CSV file.

Good for early testing at home.

Option B: Log to microSD card

More standalone.

Useful if you want others to test it in their kitchens.

Prototype development

1. Explored sensing methods

1. Mechanical arm touching the whistle/regulator
2. Microphone
3. Accelerometer/vibration sensor
4. Temperature sensor
5. Steam detection
6. Combination of multiple sensors

The mechanical arm felt reliable because it physically detects the whistle movement, but it may reduce the premium feel and may be harder to make universal across cooker designs.

So we decided to first explore a **non-contact / sensor-based version**.

2. First sensor stack

ESP32
 + INMP441 microphone
 + MPU6050 accelerometer
 + DS18B20 temperature probe
 + button
 + buzzer
 + LED

3. Started with Microphone Sensor

Goal: Read microphone data, Calculate sound energy, Log RMS and peak values, Understand what different sounds look like numerically

Microphone Calibration code to calculate

RMS = average sound energy

Peak = highest sound sample in that window

Logged the data in Python

ESP32 prints CSV data over Serial

Python reads Serial using pyserial

Python saves the data into a CSV file
 CSV can be opened in Excel or analysed in Anaconda/Jupyter

Collected and tested with various data - Human Speaking, Kitchen Noise, Pressure cooker audio from phone and then the original pressure cooker audio

Human speech can produce high and sustained RMS values. So a rule with just RMS would be dangerous.

Kitchen sounds can create very high short spikes. So peak alone is dangerous and duration tracked is useful.

Phone whistle audio was sustained but not extremely loud. This helped build early logic but is not even close to the real whistle audio.

- **First Whistle Detector (v1)**

If RMS stays above threshold for a minimum duration then count it as a whistle-like sound. This was good for the audio but not suitable for real kitchens.

Added a manual marker button, mainly used for data labeling.

The first real cooker dataset showed that the real cooker whistle was much stronger than phone audio.

Real cooker whistles produced very high RMS values.

- **Second Whistle Detector**

Updated RMS threshold and added frequency-band analysis

Main idea: Shouldn't a pressure cooker whistle have a different **frequency** signature from speech and kitchen noise?

Then the idea became - A real cooker whistle is not only loud. It also concentrates energy in certain **frequency bands**.

- **Third Whistle Detector (v1)**

Added frequency energy ratios. This tells us: How much of the sound is concentrated in whistle-like frequencies?

This gave a very clear pattern:

- During real whistles: **RMS was high, ratio_2000_6000 was high, ratio_3000_7000 was high**
- In quiet/non-whistle regions: **RMS was low, frequency ratios were mostly low**

- **Fourth Whistle Detector (v2)**

Algorithm: **RMS threshold + frequency ratio threshold + minimum duration + cooldown**

Cleaned the CSV log output.

The new algorithm detected 4 whistles and the manual buttons also marked 4 whistles. The delay b/w the manual marker and the algorithm detection was consistently around 760ms

The latest data also showed:

- Some moments had frequency ratio passing, but RMS was too low, so they were rejected.

- Some moments had high RMS but the high-frequency ratio was not correct, so they were rejected.

Validated that frequency helps reject false positives.

- **Fifth Whistle Detector (v3)**

Reduce unnecessary computation. Earlier, the ESP32 calculated frequency ratios for every audio window, which was slow. In the optimised version, the code first performs a lightweight **RMS prefilter**. Only if the RMS crosses a lower prefilter threshold does it run the heavier frequency analysis.

Current working detection logic:

1. Read audio from INMP441
2. Calculate RMS and peak
3. Check RMS prefilter
4. Calculate frequency ratios if passed
5. Check final RMS threshold
6. Check whistle-like frequency ratios
7. Confirm signal duration
8. Apply cooldown
9. Count whistle

Important Technical Lessons Learned:

1. **Phone whistle data is not enough**

Phone audio helped us start, but real cooker data is different.
Real cooker data is now our main source of truth.

2. **RMS alone is risky**

Speech and kitchen noise can create high RMS values.

3. **Peak alone is worse**

Kitchen clanks can produce huge peaks.

4. **Duration helps**

Real whistles last longer than many kitchen spikes.

5. **Frequency ratio is valuable**

Real cooker whistles show stronger energy concentration in whistle-like bands.

6. **Manual marker button was very useful**

It allowed us to compare algorithm detection against human ground truth.

7. **The current mic-only detector is promising**

It detected 4 out of 4 whistles in the latest run.

8. **We still should not rely only on sound for the final product**

For a safe, reliable kitchen product, the next step should be sensor fusion.